Introducción a UML

Diagramas de clases

Introducción a UML

- UML significa **Unified Modeling Language** (en español, **Lenguaje Unificado de Modelado**).

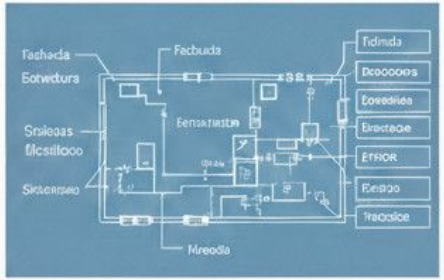
- Un **modelo** es una abstracción de un sistema o entidad del mundo real.

- Una **abstracción** es una simplificación, que incluye sólo aquellos detalles relevantes para algún determinado propósito.

- El modelado permite abordar la complejidad de los sistemas.

QUÉ ES UN MODELO

Sistema



Abstración (Simplificación)

EL VALOR DEL MODELADO

Complejidad

Modelado = Solución

Historia del UML

- Finales de los 80 - 1993: Predominio de la "guerra de los métodos". Grady Booch desarrolla el método Booch y James Rumbaugh crea OMT (Object Modeling Technique). Se desarrollan métodos diferentes, que no se ponen de acuerdo en diferentes empresas.

- 1994: Rumbaugh se une a Rational Software. Comienza la unión del método Booch y OMT.

- 1995: Nace el método unificado 0.8

- 1996: Un año después se les unió Jacobson y el método unificado se transformó en **UML** (The Unified Modeling Language).

Historia del UML

- 1997: Se presenta UML 1.0 en enero y la versión 1.1 es aceptada como estándar por la OMG (Object Management Group) en noviembre.

- 1998-2003: Evolución de la serie 1.x (1.3, 1.4, 1.5) perfeccionando el estándar.

- 2005: Se adopta **UML 2.0** por la OMG, una revisión mayor que mejora el enfoque orientado a objetos y añade nuevos diagramas.

- 2012-2017: Se lanza UML 2.5 (2012) y posteriormente la versión 2.5.1 (2017), consolidando la estructura actual.

Historia del UML

UML nació de la unión de tres metodologías dominantes creadas por ingenieros de **Rational Software**:

- **Grady Booch**: Creador del método Booch, **enfocado en el diseño**.
- **James Rumbaugh**: Desarrollador de la **Técnica de Modelado de Objetos (OMT)**.
- **Ivar Jacobson**: Creador de los métodos OOSE y Objectory, introduciendo los **diagramas de casos de uso**.

Qué es UML

- Es un **lenguaje** de modelado visual de propósito general orientado a objetos. Impulsado por el Object Management Group www.omg.org

- Un lenguaje **proporciona un vocabulario y una reglas** para permitir una comunicación. En este caso, este lenguaje se centra en **la representación gráfica de un sistema**.

- Este lenguaje nos indica cómo crear y leer los modelos, pero no dice cómo crearlos.

Qué es UML

UML es un lenguaje de modelado visual que sirve para:

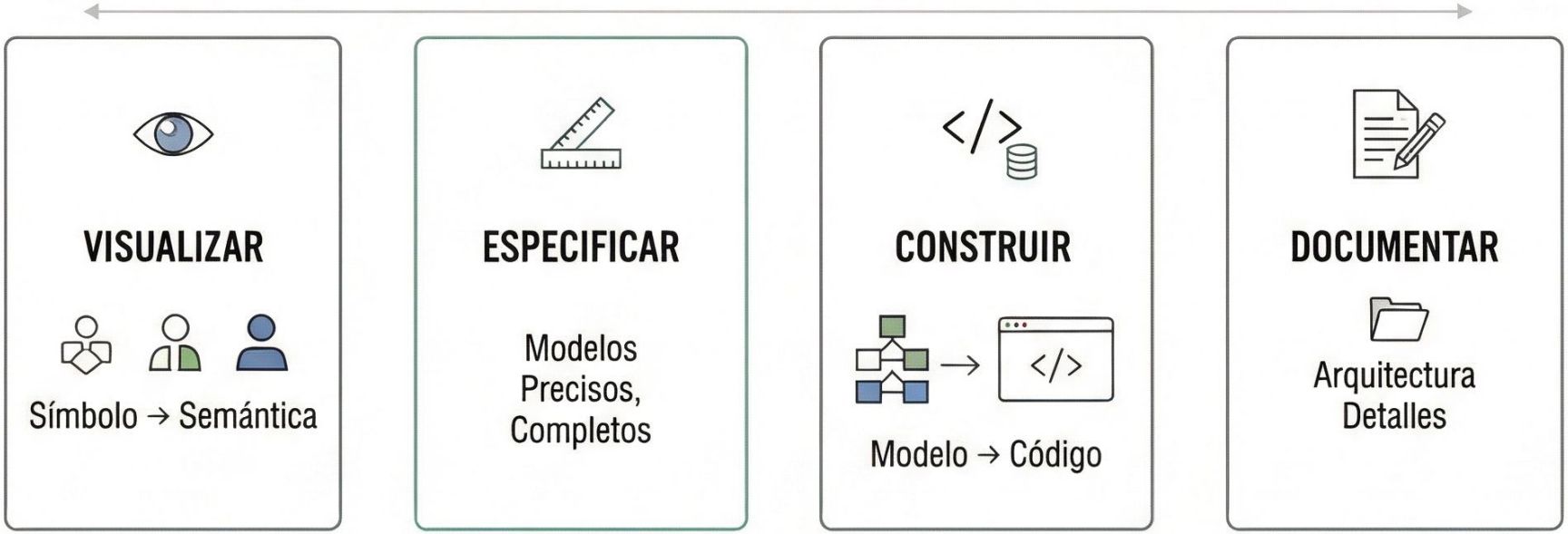
1. **Visualizar**: Detrás de cada símbolo en UML hay una semántica bien definida.

2. **Especificar**: Especificar es equivalente a construir modelos precisos, no ambiguos y completos.

3. **Construir**: Es posible establecer correspondencias entre un modelo UML y lenguajes de programación.

4. **Documentar**: UML cubre toda la documentación de un sistema, como puede ser la arquitectura de un sistema y sus detalles (diseño).

UML: LENGUAJE DE MODELADO VISUAL



Tipos de diagramas UML

A. Diagramas Estructurales: representan la "estática" del sistema (qué elementos hay).

- **Diagrama de Clases:** Estructura de las clases y su relación.

- Diagrama de Objetos: Instancias específicas de las clases con sus valores.

- Diagrama de Componentes: Cómo se organiza el código en módulos.

- Diagrama de Despliegue: Muestra el hardware y la topología del sistema.

Tipos de diagramas UML

B. Diagramas de Comportamiento: representan la "dinámica" (cómo se mueve o reacciona el sistema).

- Diagrama de Casos de Uso: Qué hace el sistema desde el punto de vista del usuario.

- Diagrama de Secuencia: Cómo interactúan los objetos a lo largo del tiempo.

- Diagrama de Actividades: Flujo de trabajo (parecido a un diagrama de flujo).

- Diagrama de Estados: Ciclo de vida de un objeto.

TIPOS DE DIAGRAMAS UML 2.5

UML 2.5: TIPOS DE DIAGRAMAS

A. DIAGRAMAS ESTRUCTURALES

Representan la "**estática**" del sistema (qué elementos hay)

- **Diagrama de Clases:** Estructura de las clases y su relación.
- **Diagrama de Objetos:** Instancias específicas de las clases con sus valores.
- **Diagrama de Componentes:** Cómo se organiza el código en módulos.
- **Diagrama de Despliegue:** Muestra el hardware y la topología del sistema.

B. DIAGRAMAS DE COMPORTAMIENTO

Representan la "**dinámica**" (cómo se mueve o reacciona el sistema)

- **Diagrama de Casos de Uso:** Qué hace el sistema desde el punto de vista del usuario.
- **Diagrama de Secuencia:** Cómo interactúan los objetos a lo largo del tiempo.
- **Diagrama de Actividades:** Flujo de trabajo (parecido a un diagrama de flujo).
- **Diagrama de Estados:** Ciclo de vida de un objeto.

Diagramas de clases: 1. La clase

- Un **Diagrama de Clases UML** es la representación visual de la **estructura estática** de un sistema.

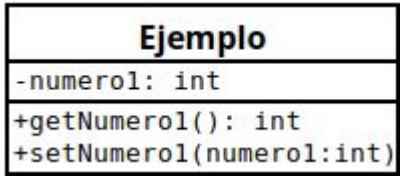
La clase. En el diagrama, una clase se representa como un rectángulo dividido en tres áreas:

- Superior: Nombre de la clase (en negrita y centrado).

- Medio: Atributos (características/datos).

- Inferior: Operaciones o Métodos

  (comportamientos/funciones).



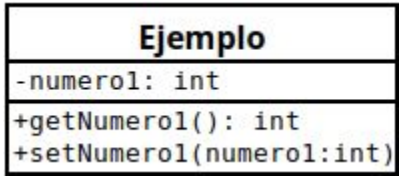
Diagramas de clases: 2. Visibilidad

2. Visibilidad: Usamos símbolos para indicar quién puede ver qué:

- Público(+): Accesible desde cualquier clase.

- Privado(-): Solo accesible dentro de la propia clase.

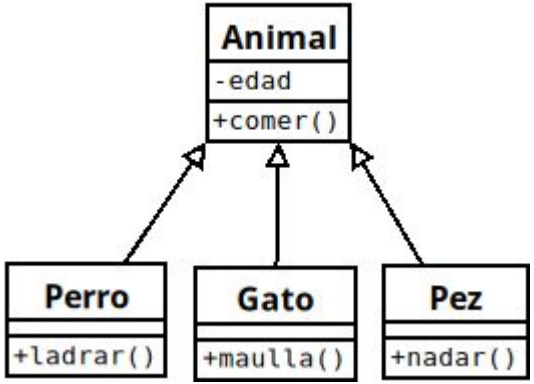
- Protegido(#): Accesible por la clase y sus hijas (herencia).



Diagramas de clases: 3. Relaciones entre clases

Se basan en los **tipos de herencia en java**. Pueden ser **herencia vertical u horizontal**. También se tiene en cuenta si la clase es abstracta o si se implementa una interfaz.

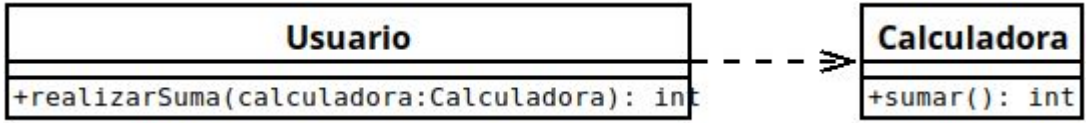
- Herencia vertical: Una clase hereda de otra. En el código se usa la nomenclatura '*extends*'.



Diagramas de clases: 3. Relaciones entre clases

- **Herencia horizontal**: Una clase usa a otra. Dependiendo del tipo de uso que le de, la relación entre las clases toma diferentes nombres:

- **Dependencia**: fuerza entre clases muy baja. Una clase usa a otra un momento.

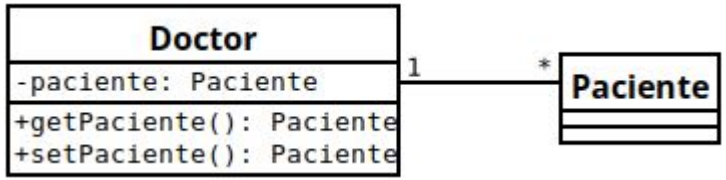


Diagramas de clases: 3. Relaciones entre clases

- **Asociación**: la relación básica de "conexión". Por ejemplo, un Doctor trata a un paciente.

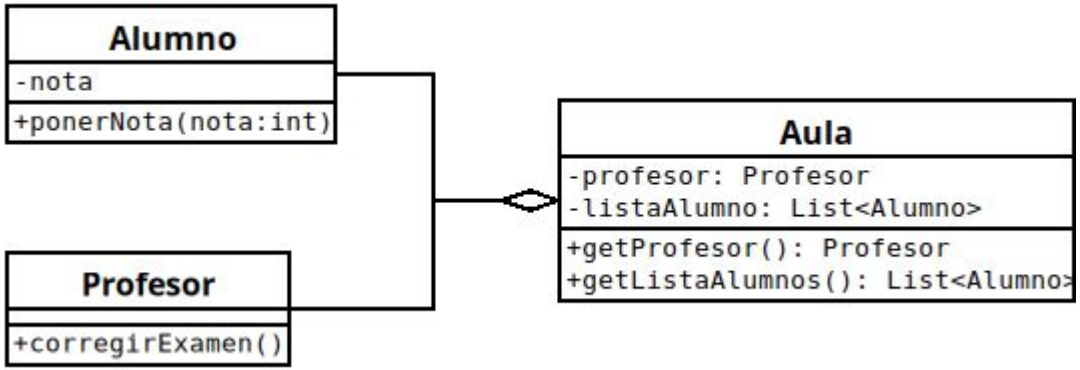
- Se pone en los extremos la **multiplicidad**, que es el rango de instancias de una clase que participan en la relación.

- `0..*`: cero o más
- `1..*`: uno o mas
- `0..1`: ninguno o solo uno
- `m..n`: al menos m, como máximo n (m<=n)



Diagramas de clases: 3. Relaciones entre clases

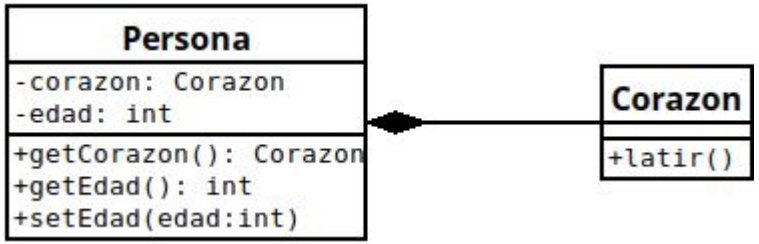
- **Agregación:** la relación representa un todo y sus partes. Las partes pueden existir sin el todo.



Diagramas de clases: 3. Relaciones entre clases

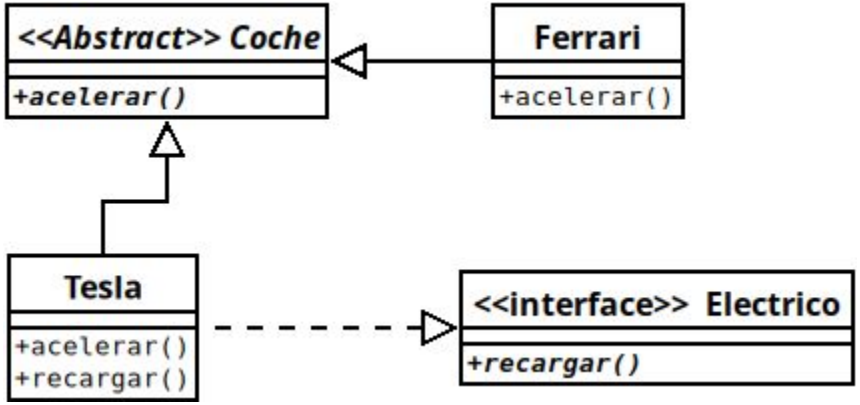
- **Composición:** la relación representa un todo y sus partes. Las partes no pueden existir sin el todo.

Ej: persona con brazo, cabeza, etc.…



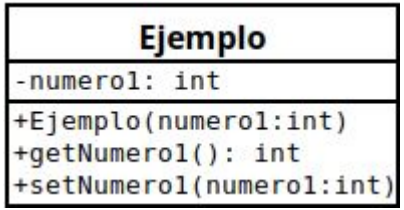
Diagramas de clases: 3. Relaciones entre clases

- **Realización/implementación:** relación entre interfaces(también clases abstractas) y las clases



Diagramas de clases: 4. Paso a código

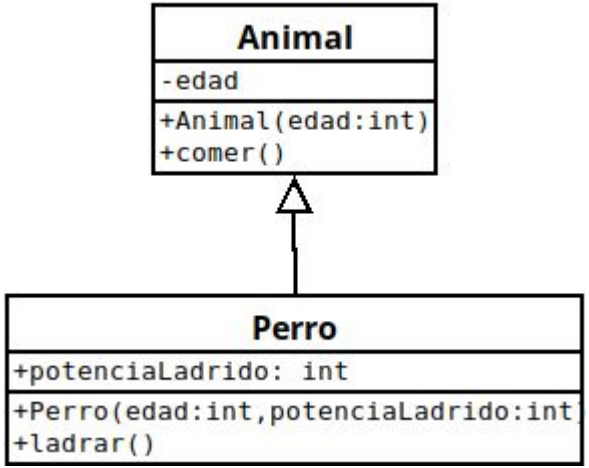
- **Clase ejemplo**




```java
//clase codigo
public class Ejemplo {
    // Variable de instancia privada (encapsulamiento)
    private int numero1;

    // Constructor para inicializar la clase
    public Ejemplo(int numero1) {
        this.numero1 = numero1;
    }

    // Getter: Permite obtener el valor de la variable
    public int getNumero1() {
        return numero1;
    }

    // Setter: Permite modificar el valor de la variable
    public void setNumero1(int numero1) {
        this.numero1 = numero1;
    }
}
```

Diagramas de clases: 4. Paso a código

- **Relación de herencia**

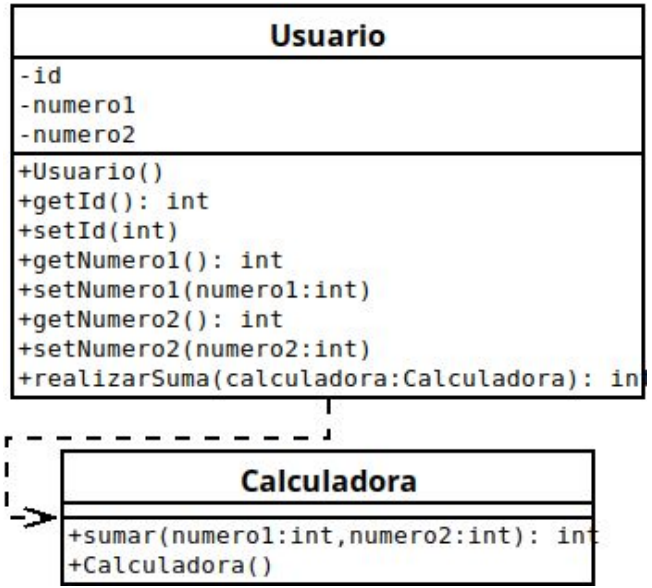



```java
//herencia
public class Animal {
    private int edad;

    // Constructor
    public Animal(int edad) {
        this.edad = edad;
    }

    // Getter
    public int getEdad() {
        return edad;
    }

    // Setter
    public void setEdad(int edad) {
        this.edad = edad;
    }
}
```

```java
public class Perro extends Animal {
    private int potenciaLadrido;

    // Constructor
    public Perro(int edad, int potenciaLadrido) {
        super(edad); // Llamada al constructor de Animal
        this.potenciaLadrido = potenciaLadrido;
    }

    // Getter
    public int getPotenciaLadrido() {
        return potenciaLadrido;
    }

    // Setter
    public void setPotenciaLadrido(int potenciaLadrido) {
        this.potenciaLadrido = potenciaLadrido;
    }
}
```

Diagramas de clases: 4. Paso a código

- **Relación de dependencia**



Diagramas de clases: 4. Paso a código

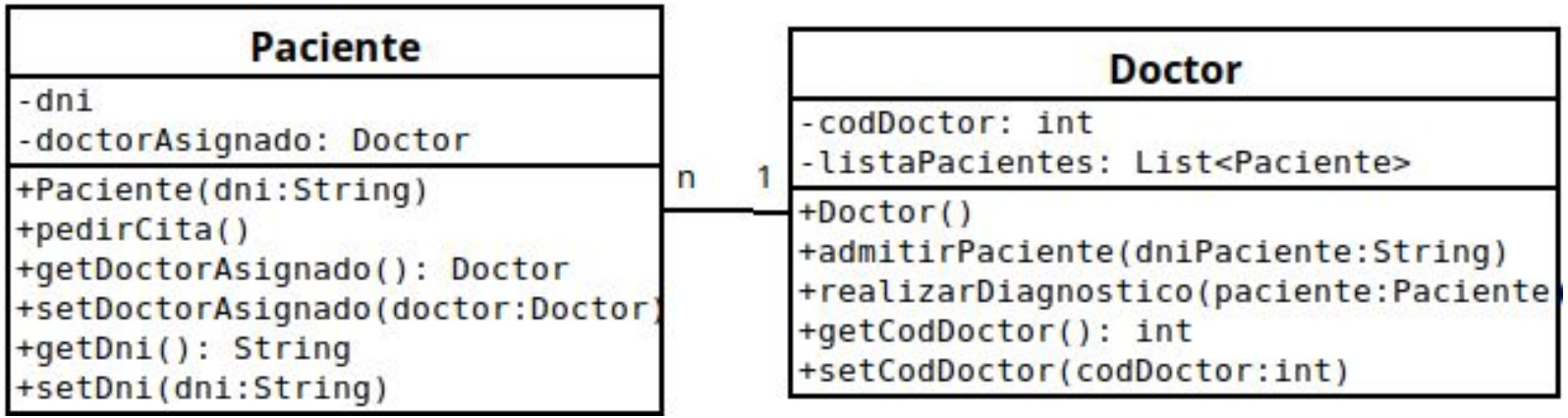
- **Relación de dependencia**

```java
//Dependencia
public class Calculadora {

    // Constructor vacío (explícito según el diagrama)
    public Calculadora() {
    }

    // Método que realiza la operación
    public int sumar(int numero1, int numero2) {
        return numero1 + numero2;
    }
}
```

```java
public class Usuario {
    private int id;
    private int numero1;
    private int numero2;

    // Constructor
    public Usuario() {
    }

    // Método de dependencia: recibe un objeto de tipo Calculadora
    public int realizarSuma(Calculadora calculadora) {
        return calculadora.sumar(this.numero1, this.numero2);
    }

    // Getters y Setters
    public int getId() {
        return numero1;
    }

    public void setId(int id) {
        this.id = id;
    }
// getters y setters de numero1, numero2......
}
```

Diagramas de clases: 4. Paso a código

- **Asociación**:



Diagramas de clases: 4. Paso a código

- **Asociación**:

```java
//asociacion
public class Paciente {
    private String dni;
    private Doctor doctorAsignado;

    public Paciente(String dni) {
        this.dni = dni;
    }

    // Método para pedir cita
    public void pedirCita() {
        if (doctorAsignado != null) {
            System.out.println("Paciente " + dni
    + " pidiendo cita con el Dr. " + doctorAsignado.getCodDoctor());
        } else {
            System.out.println("El paciente "
    + dni + " no tiene un doctor asignado todavía.");
        }
    }

    // Getters y Setters...............
}
```

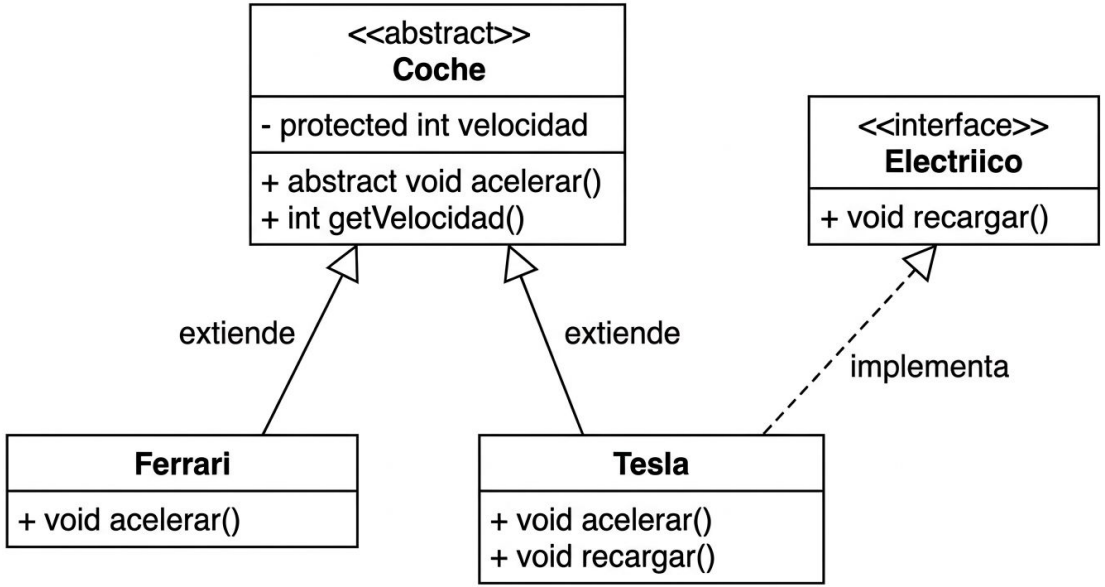
Diagramas de clases: 4. Paso a código

- **Asociación**:

```java
public class Doctor {
    private String codDoctor;
    private List<Paciente> listaPacientes; // Aquí gestionamos los "muchos" pacientes

    public Doctor(String codDoctor) {
        this.codDoctor = codDoctor;
        this.listaPacientes = new ArrayList<>();
    }

    // Método para añadir un paciente a su cargo
    public void admitirPaciente(Paciente paciente) {
        this.listaPacientes.add(paciente);
        paciente.setDoctorAsignado(this); // Establecemos la relación bidireccional en el objeto
    }

    // Método para realizar diagnóstico
    public void realizarDiagnostico(Paciente paciente) {
        if (listaPacientes.contains(paciente)) {
            System.out.println("El Dr. " + codDoctor
    + " está diagnosticando al paciente con DNI: " + paciente.getDni());
        } else {
            System.out.println("Este paciente no pertenece a la consulta del Dr. " + codDoctor);
        }
    }
    // Getters y setters......
}
```

Diagramas de clases: 4. Paso a código

- **Realización/implementación:**



Diagramas de clases: 4. Paso a código

- **Realización/implementación:**

```
// Interfaz: Define una capacidad
interface Electrico {
    void recargar();
}

class Ferrari extends Coche {
    @Override
    public void acelerar() {
        this.velocidad += 50;
        System.out.println("Ferrari rugiendo... Velocidad actual: "
     + velocidad + " km/h");
    }
}
```

Diagramas de clases: 4. Paso a código

- **Realización/implementación:**

```
abstract class Coche {
    protected int velocidad;
// protected para que las hijas accedan fácilmente

    public abstract void acelerar();

    public int getVelocidad() {
        return velocidad;
    }
}
```

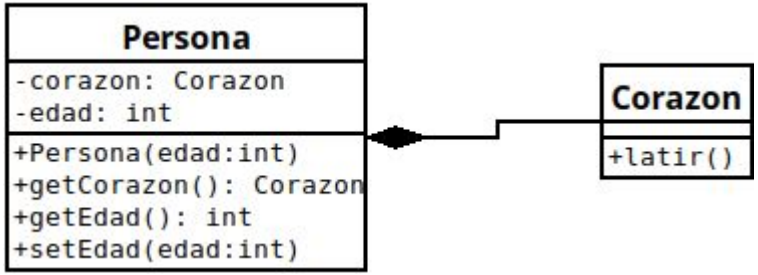
Diagramas de clases: 4. Paso a código

- **Realización/implementación:**

```
class Tesla extends Coche implements Electrico {

    @Override
    public void acelerar() {
        this.velocidad += 40;
        System.out.println("Tesla acelerando en silencio... Velocidad actual: "
    + velocidad + " km/h");
    }

    @Override
    public void recargar() {
        System.out.println("Recargando batería de litio del Tesla...");
    }
}
```

Diagramas de clases: 4. Paso a código

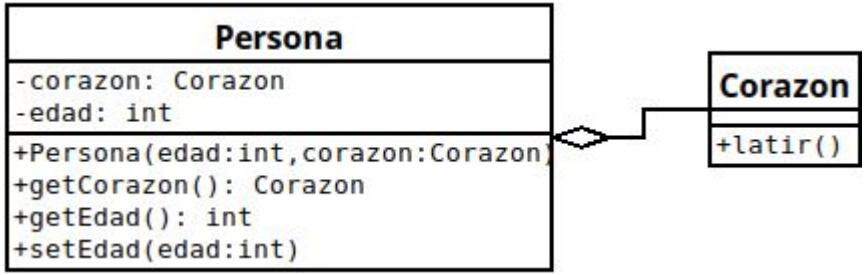
- **Composición:**




```java
//composicion
public class Persona {
    private Corazon corazon;
    private int edad;

    // Constructor: Aquí se cumple la Composición
    public Persona(int edad) {
        this.edad = edad;
        // La instancia se crea internamente
        this.corazon = new Corazon();
    }
//getters y setters
}
```

Diagramas de clases: 4. Paso a código

- **Agregacion:**




```java
//agregacion
public class Persona {
    private Corazon corazon;
    private int edad;

    // Constructor
    public Persona(int edad, Corazon corazon) {
        this.edad = edad;
        // La instancia se crea internamente
        this.corazon = corazon;
    }
//getters y setters
}
```